

Python Topic 2 — Strings & String Operations

Full Stack Python + AI Roadmap • Taught in 3 layers: New to Python → Intermediate → Expert



Layer 1 — New to Python

A **string** is just text — letters, words, sentences — wrapped in quotes.

```
name = "Akshay"
greeting = 'Hello'
```

Single `'` or double `"` quotes both work; pick one and stay consistent.

You can join strings and measure their length:

```
first = "Akshay"
last = "Chavhan"
full = first + " " + last      # "Akshay Chavhan" (concatenation)
print(len(full))              # 14 (counts every char, incl. the space)
```

The cleanest way to drop variables into text is an **f-string** — an `f` before the quote, variables in `{braces}`:

```
age = 26
print(f"My name is {name} and I am {age} years old.")
# My name is Akshay and I am 26 years old.
```



Layer 2 — Intermediate (real code + how it works)

f-strings (your main tool — like JS template literals)

```
// JS
const msg = `Hello ${name}, you are ${age}`;

# Python
msg = f"Hello {name}, you are {age}"
```

You can run real expressions inside the braces:

```
price = 1999
print(f"Total: ₹{price * 1.18:.2f}")  # Total: ₹2358.82 (:.2f = 2 decimals)
```

```
print(f"Name upper: {name.upper()}")    # Name upper: AKSHAY
```

Common string methods you'll use daily

```
s = "  Hello World  "

s.strip()          # "Hello World"      → trim whitespace (JS .trim())
s.lower()          # "  hello world  " → lowercase
s.upper()          # "  HELLO WORLD  " → uppercase
s.replace("l", "L") # replace ALL occurrences
s.split()          # ['Hello', 'World'] → split into a list
"Hello".startswith("He") # True
"Hello".endswith("lo")   # True
"World" in s           # True → substring check (very handy)
```

Indexing & slicing — grab pieces by position (0-based)

```
word = "Python"
word[0]      # 'P'   → first character
word[-1]     # 'n'   → last character (negative = from the end)
word[0:3]    # 'Pyt' → chars 0,1,2 (end index excluded)
word[:3]     # 'Pyt' → from start
word[3:]     # 'hon' → to end
word[::-1]   # 'nohtyP' → reverse (classic trick)
```

Slicing syntax is `[start:stop:step]` — far more powerful than JS's `.slice()` / `.substring()`.

Joining a list into a string (reverse of split)

```
words = ["full", "stack", "dev"]
"-".join(words)    # "full-stack-dev"
" ".join(words)    # "full stack dev"
```



Reading vs Writing a Character

These look similar but are opposite operations — the `=` is the whole difference:

```
word[0]          # READ → "give me char at position 0"    ✓ allowed
word[0] = "H"    # WRITE → "replace char at position 0"   ✗ TypeError
```

On the **right** of an expression, `word[0]` reads (always fine). On the **left** of `=`, it tries to modify in place — forbidden, because strings are immutable.

```
s = "hello"
s[0]          # ✅ 'h' — reading
s[0] = "H"    # ❌ TypeError: 'str' object does not support item assignment
```

i List contrast: Lists are mutable, so the same `[0] = ...` works there. Same syntax, opposite result — purely because `str` is immutable and `list` is mutable.

```
letters = ["h", "e", "l", "l", "o"]
letters[0] = "H"    # ✅ works! ['H', 'e', 'l', 'l', 'o']
```

i JS note: JS strings are immutable too, but writing `s[0]="H"` **silently does nothing**. Python throws a loud error instead — better, because it surfaces the bug immediately.

Layer 3 — Expert (internals & gotchas)

1. Strings are immutable

You can never change a string in place — every "edit" creates a **new** string.

```
s = "hello"
s[0] = "H"    # ❌ TypeError
s = "H" + s[1:] # ✅ "Hello" — but a brand-new object
```

🔥 Performance: Building a big string with repeated `+=` in a loop makes a new object every iteration ($O(n^2)$). Use `join` instead.

```
# ❌ Slow — new string every loop
result = ""
for w in words:
    result += w

# ✅ Fast — one allocation
result = "".join(words)
```

2. What "rebuild a new string" means (garbage collection)

"Rebuilding" doesn't instantly delete the old string — it becomes **unreferenced**, and Python frees it automatically.

```
s = "hello"    # object "hello" created, s points to it
```

```
s = "H" + s[1:]    # builds NEW "Hello"; s moves to it; "hello" orphaned
```

"hello" ← nobody points here (orphaned)
s → "Hello" ← s now points here

CPython uses **reference counting**: each object tracks how many names point to it. When the count hits **0**, its memory is freed immediately — automatically, never by you.

```
s = "hello"    # "hello" ref count = 1  
s = "Hello"    # "hello" count → 0 → freed; "Hello" count = 1
```

i But not always deleted: if another name still points to the old string, it survives.

```
a = "hello"  
b = a        # both point to same "hello" (count = 2)  
a = "Hello"  # a moves away; "hello" count → 1  
print(b)     # "hello" – still alive! b kept it
```

i JS parallel: identical model — reassign and the old value becomes garbage the GC reclaims later. You never free memory manually in either language.

3. f-strings can debug with `=` (Python 3.8+)

```
x = 42  
print(f"{x=}")    # x=42 → prints name AND value. Great for debugging.
```

4. Three quote types + raw strings

```
multi = """This spans  
multiple lines"""    # triple quotes = multi-line  
  
path = r"C:\Users\new"    # r = raw: backslashes stay literal  
# without r, \n would become a newline!
```

Raw strings are essential for regex patterns and Windows paths.

5. `str` vs `bytes` (a real gotcha from JS)

Text (`str`) and raw bytes (`bytes`) are different types. Reading files or hitting APIs, you'll meet `bytes` and need `.decode()`:

```
data = b"hello"    # bytes (note the b prefix)  
text = data.decode("utf-8")    # "hello" → str  
text.encode("utf-8")    # b"hello" → back to bytes
```

You'll hit this in file I/O, HTTP responses, and hashing.

Interview Q&A Cards

Q: Why can't you do `s[0] = "H"` on a string?

A: Strings are immutable, so item assignment (writing in place) raises `TypeError`. Reading — `s[0]` — is fine. To "change" a string you build a new one, e.g. `s = "H" + s[1:]`.

Q: When you rebuild a string, is the old one deleted?

A: Not explicitly by you. The old object becomes unreferenced; CPython's reference counting frees it once its count hits 0. If another name still points to it, it stays alive.

Q: Fastest way to build a big string from parts?

A: `"".join(list_of_parts)`. Repeated `+=` is $O(n^2)$ because each concat creates a new immutable string; `join` allocates once.

Q: Difference between `str` and `bytes` ?

A: `str` is Unicode text; `bytes` is raw binary. Convert with `.encode()` (str → bytes) and `.decode()` (bytes → str). APIs and files often give you bytes.

 **Memory hook 1:** *You can always look at a string, never poke it.* Reading = looking (fine); assigning by index = poking (forbidden).

 **Memory hook 2:** *You don't delete old strings — you just stop pointing at them.* Python throws out whatever no one's holding.

✓ **One-liner for interviews:** "Python strings are immutable sequences of Unicode characters. Because they're immutable, repeated concatenation is $O(n^2)$ — prefer `str.join()`. f-strings are the modern, fastest way to format."